Verifier-Gated LLM NetOps: A Preregistered, Artifact-Backed Evaluation of Input-Sanitization and Deterministic Verification Against Adversarial Intents

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We report a preregistered paired confirmatory experiment (study_id cand-2026-03) that measured whether template-based input sanitization combined with deterministic verifier-gating (and at-most-one automated repair) changes the rate at which a deterministic validator accepts LLM-generated CLI configurations under adversarially mutated intents. The frozen run used the declared generator gpt-5-mini and deterministic_netops_validator_v5 within the hosted_netops_gvr_v1 adapter. Planned episodes = 300; completed episodes = 282; complete paired outcomes used in the primary analysis = 141. Observed per-pair acceptance: Baseline 135/141 (95.7447%); Guarded 141/141 (100.0%); paired absolute difference = 0.04255319148936165; exact two-sided McNemar p = 0.03125; 95% bootstrap percentile CI (10,000 resamples, seed = 1729) = [0.014184397163120567, 0.07801418439716312]. We reconcile preregistration language vs the formal estimand, document per-condition pipeline semantics with explicit archived-artifact references, report the protocol deviation (unset runtime sampling temperature), and discuss operational interpretation and limitations constrained by synthetic scope and single-model/validator evaluation. All principal numeric claims are supported by archived execution and analysis artifacts referenced in Methods and Results.**

I. Introduction

Generative LLMs are increasingly proposed to translate high-level operator intent into device CLI, promising automation gains for NetOps. However, operational risks remain when generated outputs violate sequencing constraints, CLI grammar, or safety invariants; adversarially crafted inputs can exacerbate these risks. Prior prototype studies show that coupling LLM generation with deterministic verification and targeted repair can reduce observable errors, but existing evaluations often rely on token- or reference-match metrics that do not directly measure deployability or acceptance by a deterministic validator [3,8,9].

This work asks a narrowly scoped, operational question amenable to preregistered inference: How effective are template-based input sanitization and deterministic verifier-gating (with at-most-one automated repair) at changing the rate at which a deterministic validator accepts LLM-generated CLI configurations when inputs are adversarially mutated? The study cand-2026-03 preregistered a single formal estimand (paired_success_rate_difference_guarded_minus_baseline) and a confirmatory paired test. Crucially, because the registered generation semantics were shared_initial_candidate, the paired comparison isolates downstream guarded-pipeline effects (sanitization, gating, permitted repair, and final validation) applied to a single shared LLM draft per intent rather than differences in initial generation.

Contributions. (1) A preregistered, artifact-backed paired experiment executed end-to-end and reconciled against the archived preregistration; (2) a precise, artifact-grounded description of per-condition pipeline semantics that demonstrates shared_initial_candidate reuse; (3) quantitative confirmatory results showing a small but statistically detectable absolute change in deterministic-validator acceptance under the guarded pipeline on a synthetic adversarial-intent bank; and (4) an explicit reconciliation of preregistration natural-language hypothesis wording with the formal estimand and a disclosure of a protocol deviation (unset sampling temperature) together with reproducibility guidance.

II. Related Work

Deterministic network-verification tools. Formal, deterministic validators such as Header Space Analysis and VeriFlow (and related NetPlumber-style approaches) are widely used to encode and rapidly check forwarding, access-control, and sequencing invariants before deployment [5–7]. These tools operate over explicit, analyzable models of packet headers or configuration effects, and therefore directly report property-level violations (reachability, isolation, rule precedence) rather than syntactic or token-level differences. That representational difference explains their suitability as deployment gates in LLM-assisted NetOps: validators can accept or reject a candidate configuration with a semantics-focused criterion, but they require explicit formalization of the intended invariants and do not automatically cover every dynamic or protocol-level safety property. Prior literature therefore treats deterministic validators as precise but partial oracles whose effectiveness depends on the fidelity of the encoded specifications and the scope of checked properties [5–7,14].

LLM-assisted configuration and evaluation gaps. Recent empirical work that studies LLMs for NetOps (e.g., NetConfEval, NetLLMBench, and zero-touch proposals) consistently finds that model-only generation produces both syntactic errors

and higher-level semantic mistakes (ordering, precondition violations, protected-interface breaches) when translating intent to CLI [3,8,9]. Many published evaluations focus on string- or reference-match metrics that measure surface similarity to a gold command sequence; such metrics correlate poorly with deployability in practice because they neither verify execution-order constraints nor check reachability/forwarding invariants. Surveys of AIOps/telecommunications emphasize this fragmentation in evaluation practices and call for operationally grounded benchmarks that measure deployability, safety, and validator acceptance rather than token overlap alone [4,9].

Generate–validate–repair architectures and tool-assisted orchestration. A common mitigation pattern is the generate–validate–repair architecture: an LLM produces a candidate, an automated verifier checks it, and feedback is converted into constrained repair prompts or corrective actions that produce revised candidates. Prototype studies and orchestration patterns (including Toolformer-style approaches) demonstrate that integrating external checks into the generation loop improves concrete outcome metrics on narrow tasks, provided verifier feedback is sufficiently precise and repair scope is constrained [11,3]. These demonstrations show methodological promise but also reveal dependencies: (a) repair effectiveness depends on the granularity and clarity of verifier diagnostics (e.g., whether the checker returns a parsable, actionable violation code vs a high-level failure label), and (b) bounded repair policies (limited number of automatic repair attempts and explicit constraints on allowable edits) are often necessary to avoid uncontrolled changes that could mask adversarial manipulations.

Security and adversarial inputs in LLM-integrated NetOps. The LLM-systems security literature documents prompt-injection and indirect prompt-attacks that can subvert downstream behavior when external or user-supplied text is trusted by an orchestrator [10]. These attack modes motivate operational defenses used in practice, such as template-based input sanitization, slot validation, and gating designs that deterministically block candidates failing explicit safety checks. Prior defensive evaluations are often performed on synthetic or constrained datasets and may not exhaustively explore multi-source or retrieval-augmented pipelines, leaving an open need for NetOps-specific threat-modeling and rigorous mitigation studies.

Benchmarks, measurement, and operational desiderata. Several benchmark efforts and proposals (NetConfEval, NetLLM-Bench, and related task suites) attempt to exercise common NetOps intents but differ in goals: some emphasize generation diversity, others target protocol-specific behaviors. Across these efforts a recurring limitation is the lack of standardized, validator-centered deployability metrics. This paper situates itself against those gaps by preregistering an estimand that uses deterministic-validator acceptance as the primary quantity of interest and by explicitly using shared_initial_candidate semantics so that post-generation pipeline elements (sanitization, gating, bounded repair) are the causal factors under test. Unlike many prior demonstrations, the present study emphasizes artifact-backed traceability (shared-generation files, per-episode validator traces, and provider-call accounting) to connect observed numeric differences to concrete pipeline actions and to diagnose discordant cases traceable to permitted repairs.

Synthesis and remaining limitations in prior work. In sum, prior work establishes three relevant points: (1) deterministic validators provide semantically meaningful acceptance criteria but require explicit property specification; (2) generate–validate–repair architectures can reduce observable errors but their safety depends on verifier diagnostic quality, repair scope, and gating policies; and (3) existing evaluations often use token-level metrics or nonstandardized benchmarks that do not directly measure deployability under an explicit danger model. These findings motivate the present preregistered, artifact-backed paired experiment that isolates post-generation safeguards under shared_initial_candidate semantics and records per-episode validator traces to support interpretive diagnostics.

III. Methodology

Preregistration and estimand. The study cand-2026-03 was preregistered prior to observing outcomes (preregistration_sha256: ba6ed25e84c7240fbe94f2d50b3e778fbfffd810ff2db632df8946e85f0f1373). The primary estimand is paired_success_rate_difference_guarded_minus_baseline: for each adversarial intent we define a binary outcome equal to 1 if deterministic_netops_validator_v5 would accept the final candidate for deployment and 0 otherwise. The preregistered confirmatory test is an exact two-sided McNemar on paired outcomes. Bootstrap percentile 95% CIs use 10,000 resamples with seed = 1729 (analysis/analysis_log.jsonl).

Design and task bank. The preregistered design targeted N = 150 adversarial intents (paired episodes across Baseline and Guarded). The synthetic intent templates exercise interface admin changes, MTU and VLAN updates, constrained required sequences (e.g., shutdown–change–restore patterns), and minimal protected-interface rules. The adversary model used deterministic mutation heuristics applied to templates; generation seeds and mutation taxonomy are archived in execution/task_manifest.jsonl. Scope is limited to these synthetic templates and the deterministic CLI grammar encoded in our validator; no private production data was used.

Model, validator, and orchestration. The declared generator is gpt-5-mini and the deterministic validator is deterministic_netops_validator_v5 (execution/model_configuration.json). Orchestration used the hosted_netops_gvr_v1 adapter implementing a generate–validate–repair workflow. The guarded branch applied a template-based input-sanitizer prior to gating; the sanitizer and gating policy are recorded in the task manifest payloads. The archived model_configuration.json records declared_model_version = "gpt-5-mini" and an unset temperature field (temperature = null); this is disclosed as a protocol deviation (see Disclosure and Reproducibility below).

Pipeline timeline and shared_initial_candidate semantics (artifact-grounded). Because generation semantics were registered as shared_initial_candidate,

the archived execution followed this per-pair timeline (artifact references in execution/execution_manifest.json, execution/task_manifest.jsonl, and execution/responses/*-shared-initial.txt): 1) Task instantiation and adversarial mutation (execution/task_manifest.jsonl). The sanitizer output and any slot changes are recorded in the task payload when sanitization altered required fields. 2) Shared initial generation: a single LLM call produced the shared_initial_candidate for the pair and was saved to execution/responses/task-<id>-shared-initial.txt. Evidence: stage_counts.shared_initial_generation = 150 in analysis/deterministic_reconciliation.json and per-episode shared_initial_candidate files referenced by scoring artifacts (e.g., execution/scoring/task-000001-baseline.json and execution/scoring/task-000001-guarded.json). 3) Baseline scoring: deterministic_netops_validator_v5 evaluated the shared_initial_candidate; baseline scoring artifacts include validation_before and validation_after snapshots. 4) Guarded branch: template-based sanitization was applied prior to gating; the verifier-gate accepted the sanitized candidate or, if permitted by policy, allowed at-most-one automated repair LLM call. Repair-stage call traces are present for episodes that invoked repair; analysis/deterministic_reconciliation.json records repair stage_count = 6 and provider-call traces exist for those episodes. 5) Final validation: the validator re-evaluated the repaired or sanitized candidate and recorded the final guarded decision and validator trace. Representative artifacts include execution/scoring/task-000001-guarded.json which show validation_before and validation_after objects.

Missingness, exclusions and handling of technical failures. The preregistered missingness_plan specified excluding a paired unit only when both paired runs were technical failures. The archive reports completed_episodes = 282, failed_episodes = 18, and complete_pairs_included = 141 (analysis/missingness_summary.csv). Provider call accounting (analysis/deterministic_reconciliation.json) documents hosted_model_call_event_count = 156, completed = 147, failed = 9, and stage_counts: shared_initial_generation = 150, repair = 6. All exclusions and both-run failure counts are archived and reported in Results.

IV. Results

Execution and raw counts. The archived execution completed 282 of the planned 300 episodes; 18 episodes failed for technical reasons and were recorded per the preregistered missingness_plan (execution/execution_manifest.json). After applying the preregistered exclusion rule (exclude a paired unit only when both paired runs are technical failures), 141 complete pairs entered the primary confirmatory analysis (analysis/missingness_summary.csv: complete_pairs = 141; both_missing_pairs = 9). Provider-call accounting shows hosted_model_call_event_count = 156 total model-call events, with 147 completed and 9 failed calls; stage_counts report 150 shared_initial_generation events and 6 repair-stage events (analysis/deterministic_reconciliation.json).

Condition-level outcomes. Condition summary (analysis/condition_summary.csv) reports baseline_successes = 135 out of 141 complete pairs (baseline success rate = 0.9574468085106383) and guarded_successes = 141 out of 141 (guarded success rate = 1.0). These per-condition margins are the marginal rates of validator-accepted final candidates after the complete per-condition pipeline executed (baseline = validator applied to shared_initial_candidate; guarded = sanitizer $\pm$ permitted repair applied then validator re-evaluation).

Paired contingency and inference. The paired contingency table archived in analysis/paired_contingency_table.csv records n11 = 135 (both accepted), n10 = 6 (guarded accepted, baseline rejected), n01 = 0 (baseline accepted, guarded rejected), and n00 = 0 (both rejected). The preregistered confirmatory test is an exact two-sided McNemar test on these paired outcomes: p = 0.03125. The observed paired point estimate (guarded - baseline) = 0.04255319148936165. We computed a 95% bootstrap percentile confidence interval for the paired difference using 10,000 resamples with bootstrap_seed = 1729 (analysis/analysis_log.jsonl); the resulting CI is [0.014184397163120567, 0.07801418439716312]. All numbers and test parameters are archived and reproducible from analysis/* artifacts.

Discordant-case diagnostics and repair association. The six discordant n10 pairs (guarded=1, baseline=0) are traceable in the archived per-episode scoring artifacts. Deterministic reconciliation and provider-call accounting link repair-stage activity to these discordant pairs: stage_counts.repair = 6 (analysis/deterministic_reconciliation.json). For each discordant pair the shared_initial_candidate file (execution/responses/task-<id>-shared-initial.txt) is identical between the baseline and guarded scoring artifacts for that pair (execution/scoring/task-00000X-baseline.json and execution/scoring/task-00000X-guarded.json reference the same shared_initial_candidate_sha256). The baseline scoring artifact records validation_before/validation_after showing an initial rejection (validator.valid = false in validation_before or validator flags) while the guarded scoring artifact shows validation_after.valid = true following sanitizer and/or the single permitted repair. Representative examples are packaged as execution/scoring/task-000001-baseline.json and execution/scoring/task-000001-guarded.json and their associated shared_initial response execution/responses/task-000001-shared-initial.txt.

Interpretation of discordant repairs. The archived validator traces indicate that in these six cases the guarded pipeline converted an initial draft the validator would reject into a final candidate the validator accepted. That conversion can occur via (a) sanitizer-caused normalization or slot-filling that restored required fields, or (b) a single automated repair LLM call whose output satisfied the validator. The artifacts show repair_applied = true for the episodes that invoked repair; the scoring artifacts explicitly record repair_applied and repair_prompt_sha256 when present. Because the formal estimand measures validator acceptance-for-deployment, these discordant conversions increase the guarded acceptance rate

even though they may reflect recovered benign drafts rather than elimination of adversarial advantage. This diagnostic is central to the interpretive boundary discussed in the paper.

Missingness sensitivity and conservative accounting. Nine paired units were excluded because both runs failed technically (analysis/missingness_summary.csv: both_missing_pairs = 9; failed_baseline_episodes = 9; failed_guarded_episodes = 9). As a conservative sensitivity check (not the preregistered primary analysis), treating those excluded pairs as failures yields baseline acceptance = 135/150 = 0.9000 and guarded acceptance = 141/150 = 0.9400. This sensitivity demonstrates that technical failures reduce effective sample size but do not invert the direction of the observed paired difference under the archive's conservative assumption. All failure counts and the conservative recalculation are reproducible from analysis/missingness_summary.csv and analysis/condition_summary.csv.

Unexecuted preregistered items. The preregistered benign-control (M = 50) and the false-positive-blocking secondary estimand were not executed in this run; no benign-control artifacts are present in the task or execution manifests. Consequently no empirical false-positive-blocking rates are reported here; this omission and its practical implications are documented in the Methods, Discussion, and Limitations. The analysis artifacts explicitly show which planned secondary analyses were executed (analysis/*) and which were absent from the run manifest.

V. Discussion

Hypothesis reconciliation and interpretive boundary. The preregistration's natural-language H1 asserted that verifier-gating would reduce attacker success (i.e., fewer unsafe configurations accepted). The registered formal estimand measured paired validator acceptance-for-deployment (guarded - baseline). These constructs differ: an increase in validator acceptance can reflect successful repair of malformed but benign drafts and not necessarily increased attacker success. Our observed guarded→higher acceptance (≈4.255 percentage points) occurred because sanitizer and permitted repair recovered reparable drafts that the baseline validator rejected. Therefore the preregistered directional statement about reducing attacker success is not supported by the formal-estimand outcome; we report and interpret the archived formal estimand explicitly and caution against conflating validator acceptance with attacker-centric safety without an independent semantic-safety oracle.

Operational implications and repair–gate co-design. Within the synthetic setting, input sanitization plus bounded automated repair can reduce operator rework by converting invalid drafts into validator-accepted candidates. However, this benefit depends on the validator's semantic coverage and the scope of permitted repairs: the six discordant cases show repairs enabling acceptance, which is beneficial if repairs preserve intended semantics but risky if repairs mask adversarial manipulations. This highlights an operational co-design requirement: gating policies, repair scope, and validator expressiveness must be jointly specified to preserve safety in deployment.

Practical trade-offs illustrated by the archive. The artifact evidence (analysis/deterministic_reconciliation.json, analysis/paired_contingency_table.csv, and per-episode scoring artifacts such as execution/scoring/task-000001-guarded.json) shows precisely how small numbers of permitted repairs materially changed final acceptance. That pattern exposes a predictable trade-off: stricter gating (reject-on-violation) reduces the chance of a repaired but semantically altered candidate being accepted, at the cost of more operator interventions; more permissive policies with bounded repairs reduce operator workload but raise the need for stronger semantic checks or human review on repaired cases. These trade-offs are operational, measurable with the archived logs (repair stage_count = 6, hosted_model_call_event_count = 156) and must be chosen to reflect acceptable risk budgets in deployment.

Failure modes of generate–validate–repair loops. The archived validator traces document three related failure modes relevant to operational design: (1) reparable syntactic/ordering errors where repair restores intent-preserving structure (the desirable class observed for the six discordants); (2) repair that fills missing fields or normalizes values in ways that change semantics subtly (requires semantic-safety or reachability checks beyond the current validator); and (3) cases of technical failure or timeouts (18/300 episodes, both-run failures = 9) where neither repair nor gating completed, reducing effective sample size (analysis/missingness_summary.csv). Operational pipelines should instrument and surface these distinct failure categories to operators and governance audits using the same per-episode trace primitives present in our archive.

Reproducibility and the protocol deviation. The unset runtime sampling temperature (temperature = null in execution/model_configuration.json) is a recorded protocol deviation. Its practical implication depends on provider default behavior; because the run used shared_initial_candidate semantics (stage_counts.shared_initial_generation = 150) and provider-call accounting confirms the observed call pattern (analysis/deterministic_reconciliation.json), paired comparability for the formal estimand is preserved in this execution. Nevertheless, for deterministic reproduction we recommend setting temperature = 0 explicitly (execution/model_configuration.json) and re-running the archived task manifest; the archive provides all artifacts necessary to compare outcomes deterministically (execution/responses/*, execution/scoring/*, analysis/*).

Governance, auditing, and measurement recommendations grounded in archived evidence. The run demonstrates the value of (a) detailed per-episode validator traces (validation_before/after objects in scoring artifacts) to audit repairs; (b) explicit repair audit logs and provenance (present for repaired episodes) so that operator review can target only changed cases; and (c) a preregistered benign-control to quantify false-positive-blocking (not executed here). The archive's contamination and missingness summaries (analysis/contamination_summary.csv is empty; analy-

sis/missingness_summary.csv reports counts) show these governance metrics can be computed automatically from the same instrumentation used in this study.

Limits to operational generalization. The discussion above is tethered to the archive: a single LLM (declared gpt-5-mini), a single deterministic validator implementation, and synthetic templates. These constraints mean the measured acceptance-rate change is a within-bundle operational observation, not a claim of cross-vendor or production robustness. The ceiling effect (high baseline acceptance) reduces practical headroom for improvement; this is visible in analysis/condition_summary.csv and motivates future runs that (1) execute the preregistered benign-control to measure false positives, (2) adopt attacker-centric semantic-safety or reachability oracles in addition to syntactic validators, and (3) diversify model and validator families before making operational deployment claims.

Concluding interpretive note. The archived data concretely show that bounded verifier-gated repair can increase validator acceptance by recovering reparable drafts; whether that outcome improves or harms security depends on whether repairs preserve intended semantics—a property not fully covered by the present deterministic validator and therefore not directly measured here. The empirical artifacts (per-episode scoring traces, repair counts, and paired contingency outputs) provide the necessary operational observables to design follow-up confirmations that target attacker-centric estimands and benign-control measurements.

VI. Limitations

- Synthetic task bank and intent templates — not real production traffic or operator logs; results may not generalize to field datasets.
- Single-model evaluation (gpt-5-mini) and single validator implementation limit external validity across vendors, protocols, and model families.
- Validator coverage constrained to encoded safety properties (CLI grammar, required sequences, protected-interface invariants); some protocol-level or dynamic runtime semantics are not represented.
- Technical failures (18/300 episodes) reduced effective sample size; paired exclusion (both-run failures = 9) was preregistered and applied.
- Adversary model limited to mutations of synthetic intents; prompt-injection in retrieval-augmented or multi-source pipelines was not exhaustively evaluated.
- A preregistration natural-language hypothesis phrasing differed from the registered formal estimand; results are reported and interpreted with respect to the archived formal estimand.
- The preregistered benign-control (M = 50) and false-positive-blocking secondary estimand were not executed; therefore no empirical false-positive-blocking claims are made.

VII. Conclusion

A preregistered, artifact-backed paired experiment on a synthetic adversarial intent bank found that template-based input sanitization combined with deterministic verifier-gating and at-most-one automated repair produced a small but statistically detectable absolute increase in deterministic-validator-accepted LLM-generated configurations under the archived formal estimand (paired difference $\approx$ 0.04255; p = 0.03125; 95% CI [0.01418, 0.07801]). Diagnostics show the six discordant pairs were associated with permitted repair calls that enabled acceptance. Importantly, this formal-estimand improvement does not equate to measured reduction in attacker success under attacker-centric definitions. The run is limited by synthetic scope, a single model/validator, and a protocol deviation (unset runtime temperature). Future work should execute the preregistered benign-control, measure attacker-centric estimands with an independent semantic-safety oracle, diversify models/validators, and set explicit sampling parameters for reproducible deterministic runs. All principal numbers and reconciliation claims above are supported by archived execution and analysis artifacts referenced in Methods and Results (e.g., execution/task_manifest.jsonl; execution/responses/task-000001-shared-initial.txt; execution/scoring/task-000001-baseline.json; execution/scoring/task-000001-guarded.json; analysis/paired_contingency_table.csv; analysis/condition_summary.csv).

Disclosure Statement

This study was executed autonomously after an autonomy lock under the archived master prompt (provenance/master_prompt.txt, SHA-256: 1872df1e1805d2d9 6940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human role and autonomy boundary: a human Research Director selected the topic family (Generative AI / LLMs for NetOps) and constrained the initial master prompt prior to the autonomy lock; all literature review, preregistration, study selection, code generation, experiment execution, analysis, peer-review cycles, manuscript drafting, and revision reported here were performed autonomously after the lock. Declared models/tools and orchestration: primary generator = gpt-5-mini (execution/model_configuration.json), deterministic validator = deterministic_netops_validator_v5, task generator = deterministic_netops_task_generator_v5, orchestration adapter family = hosted_netops_gvr_v1. Preregistration: study cand-2026-03 was preregistered and archived prior to observing outcomes (preregistration_sha256: ba6ed25e 84c7240fbe94f2d50b3e778fbfffd810ff2db632df8946e8 5f0f1373). Protocol deviation: the archived run recorded an unset runtime sampling temperature (temperature = null in execution/model_configuration.json); this is disclosed as a deviation because provider-side defaults may affect sampling determinism. To preserve paired comparability the pipeline used shared_initial_candidate semantics (single shared generation per pair) and provider-call accounting documented in analysis/deterministic_reconciliation.json

confirms the actual call pattern (shared_initial_generation = 150; repair = 6). Key archived artifact references (examples): execution/task_manifest.jsonl; execution/responses/task-000001-shared-initial.txt; execution/scoring/task-000001-baseline.json; execution/scoring/task-000001-guarded.json; analysis/deterministic_reconciliation.json; analysis/paired_contingency_table.csv; analysis/condition_summary.csv; analysis/analysis_log.jsonl; execution/model_configuration.json; execution/execution_manifest.json. No human manually edited or corrected the manuscript technical content after the autonomy lock.

REFERENCES

[1] http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659
[2] https://doi.org/10.1145/2342441.2342452
[3] https://doi.org/10.1145/3626111.3628194
[4] https://doi.org/10.1145/3605764.3623985
[5] http://arxiv.org/abs/2302.04761
[6] https://doi.org/10.1145/3656296
[7] Hao Zhou, Chengming Hu, Ye Yuan, Yufei Cui, Yili Jin, Can Chen, Haolun Wu, Dun Yuan, Li Jiang, Di Wu, Xue Liu, Jianzhong Zhang, Xianbin Wang, Jiangchuan Liu, "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities", 2024, 10.1109/comst.2024.3465447
[8] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635
[9] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194
[10] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, Mario Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection", 2023, 10.1145/3605764.3623985
[11] Yupeng Chang, Xu Wang, Jindong Wang, Yuan-Hsuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, Wei Ye, Yue Zhang, Yi Chang, Philip S. Yu, Qiang Yang, Xing Xie, "A Survey on Evaluation of Large Language Models", 2023, 10.48550/arxiv.2307.03109
[12] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Răileanu, María Lomelí, Luke Zettlemoyer, Nicola Cancedda, Thomas Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools", 2023, 10.48550/arxiv.2302.04761
[13] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296
[14] Peyman Kazemian, George Varghese, Nick McKeown, "Header space analysis: static checking for networks", 2012